Tips for your report, thesis or paper

Probabilistic programming group

Thomas Hamelryck, January 2026

Design guide KU

KU provides Word and Latex templates etc. for master's theses and PhD dissertations.
You can find the DesignGuide here.

References and background reading

Below are some references for **background reading on machine learning and Bayesian statistics. Note that not everything will be equally relevant for you.**

- Machine learning 101 cheatsheet
- How to avoid machine learning pitfalls: a guide for academic researchers by Michael A. Lones
- **Bayesian statistics & Bayesian machine learning**
    - A tutorial, using Python, on the **difference between Bayesian and frequentist statistics** by Jake VanderPlas.
    - A freely available book on **probabilistic machine learning** by Chris Bishop.
    - A tutorial on neural networks as **Bayesian probabilistic models**.
    - ArXiv: How Good is the Bayes Posterior in Deep Neural Networks Really?
    - Blog: On the choice of priors by Andrew Gelman
    - ArXiv: The prior can generally only be understood in the context of the likelihood
    - Blog: A Bayesian workflow by Michael Betancourt
    - ArXiv: Bayesian workflow by Gelman et al.
- **Deep learning**
    - A book on deep learning by Goodfellow, Bengio and Courville.

    - What are neural networks really doing?
    - Recurrent neural networks, LSTMs and GRUs
    - Convolutional neural networks
    - Deep generative models should be Bayesian
    - The lottery ticket hypothesis
    - Neural network work by changing topology layer-by-layer
    - Deep learning is not so mysterious or different by Andrew Gordon Wilson (on benign overfitting, double descent, and the success of overparametrization)
- **Probabilistic programming languages** (PPLs)
    - The articles describing Pyro and NumPyro.
        - Pyro is based on PyTorch.
        - Numpyro is based on JAX.
            - Note: We use **Numpyro instead of Pyro** for some projects because Numpyro's iterative NUTS method is much faster than default NUTS.
        - Ahmad's blog on Pyro's Bayesian layers for Bayesian neural networks.
    - Thomas Wiecki's excellent blogs on probabilistic programming.
        - The blog is based on pyMC instead of Pyro, but the methods and insights are universal.
    - Statistical rethinking by Richard McElreath.
        - The book uses the PPL STAN and **R**, but the examples have been ported to Numpyro.
- Inference using **Markov Chain Monte Carlo** (MCMC)
    - An Introduction to MCMC for Machine Learning
    - Betancourt's introduction to Hamiltonian Monte Carlo (HMC) and NUTS
        - This is the MCMC method of choice for PPLs
    - Another tutorial on HMC and NUTS
- Inference using **optimization** (**Stochastic Variational Inference, SVI**)
    - An introduction to **variational autoencoders (VAEs), stochastic variational inference** and the **reparameterization trick**.
    - A much more elaborate introduction by Diederik Kinga & Max Welling, who pioneered the use of VAEs.
        - Their original paper on the variational autoencoder

    - Posterior collapse: why and how training VAEs can go wrong
        - Søren Hauberg's heuristics to make it work
    - Overview of gradient descent optimization algorithms (Adam, Adagrad, etc.)
- **Directional statistics**
    - Directional statistics is the statistics of **angles and other non-Euclidean data**, which is often relevant for protein structure prediction and analysis.
    - A recent review on directional statistics by Pewsey & García-Portugués.
    - Sampling from the bivariate von Mises distribution
    - Christophe Ley's method to skew toroidal distributions

A simple Google search will deliver many accessible **tutorial videos about the above topics and more**. The above references are **far from exhaustive**. **You are expected to explore the literature relevant to your specific tasks independently.**

Writing your paper

- Video talk on writing effectively by Larry McEnerney
- Make sure the **goal** of your paper is very clear already in the abstract. Notably consider:
    - What is the **current state of affairs** and what is **lacking**?
    - **What** am I investigating and **how** and **why**?
- **Notation for probabilistic models and neural networks**
    - The notation of the book by Goodfellow, Bengio and Courville.
    - The notation of the article by Kinga & Welling (page 67)
- Do not use references to **Wikipedia articles**.
    - You can of course use Wikipedia for informal study, exploration and locating references.
- You can use references to **arXiv** and **websites** where **no peer-reviewed alternatives** are to be found.
- Do not spell out in **many words** what can be said in a **few formulas or a picture**.
- Provide **enough mathematical detail.**
    - For example, specify the PDF of a non-standard probability distribution like the bivariate von Mises.

- Write your report in LaTeX or the easy-to-use WYSIWYG **LaTeX editor** Lyx, if at all possible.
    - If you want to **share** your LaTeX document and get comments, consider Overleaf.
    - You can also make your **slides** using LaTeX+Beamer or Lyx+Beamer
- Figures
    - For **probabilistic models**, consider Graphviz.
    - For **neural networks**, consider NN-SVG.
- Don't be **overly verbose**.
    - 30 pages for a thesis is already plenty - keep it **to the point and interesting**.
    - Project: 5-15 pages or so (not counting references).
        - Put pages beyond 8 pages in an appendix.
- **Structure** your paper as follows:
    - Use a suitable Latex arXiv template
        - Here are some alternative templates
    - Title, Authors, Affiliations (emails etc)
    - Abstract
        - Make sure the research goals are clearly stated.
    - Introduction
        - Here you explain what the point of your work is and why it is relevant.
    - Prior work
        - Here you give a deeper overview of work done in the area you are concerned with.
    - Data and methods
    - Results
    - Discussion
    - Conclusion
    - Acknowledgements
    - References
    - Appendices
- If you encounter **problems** (for example, you had trouble training a model), simply describe those problems, how you tried to address the problems and what you think is going wrong. That is, **provide a decent analysis of any problems in your paper**.

- A good way to get your paper up to standard is to select a related, published paper that you found exceptionally **clear and well written**, and **use that paper as a guide**.
    - Your thesis should read like an accessible peer-reviewed paper, with **just a bit more explanation and background than is customary** (because you want to show that you master the material in this **academic situation**).
- **Avoid plagiarism**!
    - Provide references for **any** quotation or figure you borrowed.
        - Note that this includes your own previous papers.
    - Plagiarism is **automatically checked** in Digital Exam. **We see it!**
    - Plagiarism can easily get you **expelled**! We have to report it and from there on it's out of our hands.
- Remember **labelling coordinate axes** and provide **adequate figure legends**.
    - Refer to your figures in the text.
- Do not assume the **censor will have very specialised knowledge on your topic**.
    - Make sure your thesis is **easy to read for non-specialists**.
    - Ask yourself: **would I enjoy reading my own thesis?**
- **Appealing layout**, **fonts** and **figures** matter!
- It's perfectly fine to discuss your implementation and show **relevant code snippets**.
- If you want feedback, make sure I have your paper **several weeks before the deadline**.

Good coding and working practices

- Run on a **server**, and not a laptop, so you can run for a long time without trouble.
    - Google's Colab provides a freely accessible online computing environment for deep learning, including access to GPUs.
        - A blog on running Pyro on Colab
        - Colab cheat sheet
        - **Activate** the GPU on Colab:
            - Runtime→Change runtime type→Hardware Accelerator→GPU
- Use training, validation and test sets.

    - If you don't (for example, because visual inspection is sufficient at this point), explain clearly why. See below for **prior/posterior predictive checks**.
    - Once a model is optimised using the validation set, **no more changes can be made** before running it on the test set for the final results that end up in the report!
    - The split must be reproducible (like everything you do).
        - Scikit-learn has a method to split data sets automatically
- **Evaluating model fit (ie. finding the "best" model)**
    - **SVI**
        - Use the ELBO on the test or the validation data set
    - **NUTS and MCMC**
        - Notice any divergences in NUTS.
        - Use Leave-One-Out cross-validation (LOO) and the Widely Applicable Information Criterion (WAIC)
            - ArviZ provides code for LOO and WAIC and allows import of samples from Numpyro
            - A short tutorial video on model evaluation using AIC, DIC, WAIC and LOO
        - Use prior and posterior predictive checks
            - Example in Numpyro for Bayesian linear regression
            - Another example using pyMC3
- You can **develop, test and debug your models using a small data set**.
    - Once everything works, run it on larger amounts of data.
- If you use **MCMC methods such as NUTS**, use diagnostics to evaluate convergence of the sampling.
    - See this article by Vehtari, Gelman *et al.* on state-of-the-art assessment of MCMC convergence (R-hat, tail ESS, rank plots). The links below provide implementations of what is discussed in the article.
        - Rank normalized R-hat diagnostic in ArviZ
        - Various types of ESS in ArviZ
- Put your **code** on Github so you have a **backup** and can use **version control**.
    - Here are some Github tutorials
    - Share the Github repository, so we can look at your code and comment.

        - My Github id is **thamelry**
- Make sure your experiments are **reproducible**.
    - Use random number generator seeds where necessary.
- Do run-time analysis of your algorithms where relevant
    - How does speed scale with the amount of data?
        - Provide a graph where relevant.
    - How to measure execution time in Python.
- Adequately comment your code.
- Use good Python coding style
- Consider using a **notebook**.
    - **Jupyter + Github or Colab+Github are winning combinations for getting good feedback**.
- Develop in an IDE like PyCharm (the Pro version is free for students)
    - PyCharm integrates with Github.
- Learn to use a debugger.

The exam

- **Oral defense structure masters/projects**
    - You give a 30/15 min talk with slides.
        - I recommend making the **slides** using LaTeX+Beamer or Lyx+Beamer
        - Make sure to **number your slides**
    - This is followed by 30/15 min questioning by the censor and supervisor.
    - The aim is to have an **interesting scientific discussion about your work** and evaluate **how well you understand what you did**.
    - Make sure you can **explain ALL your models and inference algorithms conceptually.**
    - Defence over **Zoom** is possible.
- Questions that can be expected
    - General questions
        - Details about the model, hyperparameters, choice of priors,...
        - Details on model selection and testing
        - Computational complexity and speed of your model

        - What is the number of parameters of your model?
        - Why is NUTS/SVI/MAP/etc. the right inference method for your model?
    - If you work with **deep learning / deep probabilistic programming**
        - What is probabilistic programming?
        - What is deep probabilistic programming?
        - What is the difference between RNNs, LSTMs and GRUs?
        - What is automatic differentiation and why is it important?
    - If you work with **SVI**
        - Explain SVI
        - Why do we use SVI? Why not just MAP?
        - What is the ELBO?
        - What is posterior collapse in VAEs?
        - Explain the optimization method used
        - What is the reparameterization trick?
        - What is amortised inference?
        - What is a Guide and a Model?
        - What is the difference between an autoencoder and a variational autoencoder?
        - What is the probabilistic model behind a variational autoencoder?
        - Are VAEs fully Bayesian? Discuss.
        - Can SVI be used for large data problems? Why or why not?
    - If you work with **MCMC / NUTS**
        - Explain HMC
        - What is the difference between HMC and NUTS?
        - What is the difference between NUTS and iterative NUTS?
        - How did you evaluate convergence?
            - Diagnosis of MCMC chain quality
        - How did you decide on the number of samples used?
        - Can NUTS be used for large data problems? Why or why not?